



**iSMAGi**

Institut Supérieur de Management  
d'Administration et de Génie Informatique  
المعهد العالي للتدبير و الإدارة و الهندسة المعلوماتية



المركز الإستشفائي الجامعي ابن سينا  
مركز المستشفى الجامعي ابن سينا  
CENTRE HOSPITALO-UNIVERSITAIRE IBN SINA

## **RAPPORT DE PROJET DE FIN D'ÉTUDES**

**Tableau de Bord Décisionnel pour le CHU Ibn Sina – Urgences Hospitalières**



**iSMAGi**

Institut Supérieur de Management  
d'Administration et de Génie Informatique  
المعهد العالي للتدبير و الإدارة و الهندسة المعلوماتية



المركز الإستشفائي الجامعي ابن سينا  
مرکز بیمارستانی دانشگاه ابن سینا  
CENTRE HOSPITALO-UNIVERSITAIRE IBN SINA

## REMERCIEMENTS

Je tiens à remercier chaleureusement toutes les personnes ayant contribué à la réalisation de ce projet de fin d'études.

Mes sincères remerciements s'adressent à l'ensemble du personnel du Centre Hospitalier Universitaire Ibn Sina de Rabat pour leur accueil, leur disponibilité et leur accompagnement durant cette période de stage.

Je remercie particulièrement mon encadrant professionnel ainsi que mon encadrant pédagogique pour leurs conseils, leur suivi et leur soutien tout au long du projet.

Je remercie également ma famille et mes amis pour leur encouragement et leur soutien moral.



## DÉDICACE

Je dédie ce travail :

- À mes chers parents pour leur soutien inconditionnel.
- À ma famille pour leurs encouragements permanents.
- À mes amis et collègues pour leur aide et leur motivation.
- À toutes les personnes qui ont contribué de près ou de loin à la réussite de ce projet.



## RÉSUMÉ

**Contexte :** Face à la croissance des données hospitalières et à la nécessité d'améliorer la prise de décision, le CHU Ibn Sina de Rabat a besoin d'un outil intelligent de Business Intelligence (BI) pour ses services d'urgences.

**Objectif :** Concevoir et développer un système décisionnel complet intégrant :

- Une centrale de données multi-établissements (HIS, HMY, HSR, HEY, CSR, HAR, MAT).
- Un pipeline ETL automatisé.
- Un tableau de bord interactif (Power BI + React.js).
- Des modèles prédictifs (Prophet, Random Forest, XGBoost) pour l'affluence et la durée de séjour.

### Résultats clés :

- Centralisation de 1,2 million d'enregistrements (simulations).
- Pipeline ETL avec architecture Medallion (Bronze, Silver, Gold).
- API REST FastAPI avec 15 endpoints, authentification JWT.
- Dashboard avec 12 KPI dynamiques.
- Modèle XGBoost :  $R^2 = 96,70\%$  pour la prédiction de durée de séjour.
- Modèle Prophet :  $R^2 = 95,57\%$  pour la prévision d'affluence à 7 jours.

**Impact :** Aide à l'optimisation des ressources, réduction des temps d'attente, anticipation des pics de fréquentation.

**Mots-clés :** Business Intelligence, ETL, Machine Learning, Dashboard, Urgences, CHU, FastAPI, React, Docker.



## Table des matières

|                                                              |    |
|--------------------------------------------------------------|----|
| REMERCIEMENTS .....                                          | 2  |
| DÉDICACE.....                                                | 3  |
| RÉSUMÉ.....                                                  | 4  |
| Liste des Figure : .....                                     | 7  |
| Liste des Tableaux : .....                                   | 8  |
| INTRODUCTION GÉNÉRALE.....                                   | 9  |
| CHAPITRE 1 : CONTEXTE GÉNÉRAL DU STAGE .....                 | 10 |
| 1.1 Introduction .....                                       | 10 |
| 1.2 Présentation de l'organisme d'accueil.....               | 10 |
| 1.3 Présentation du projet.....                              | 11 |
| 1.3.1 Objectifs SMART .....                                  | 11 |
| 1.3.2 Livrables attendus .....                               | 12 |
| 1.3.3 Équipe et rôles .....                                  | 12 |
| 1.4 Outils et technologies : .....                           | 13 |
| 1.5 Contraintes et difficultés rencontrées – Solutions ..... | 13 |
| 1.6 Indicateurs de performance (KPI) .....                   | 14 |
| 1.7 Méthodologie du projet (cycle en V adapté) .....         | 14 |
| Phases avec jalons : .....                                   | 15 |
| 1.8 Conclusion.....                                          | 15 |
| CHAPITRE 2 : PIPELINE ETL.....                               | 16 |
| 2.1 Introduction .....                                       | 16 |
| 2.2 Architecture Medallion : .....                           | 16 |
| 2.3 Sources de données – Description détaillée : .....       | 17 |
| 2.4 Étapes détaillées du pipeline .....                      | 17 |
| 2.4.1 Extraction (Extract) .....                             | 17 |
| 2.4.2 Transformation (Transform).....                        | 18 |
| 2.5 Automatisation et orchestration.....                     | 20 |
| 2.6 Tests et validation du pipeline .....                    | 20 |
| 2.7 Conclusion.....                                          | 20 |



|                                                                         |    |
|-------------------------------------------------------------------------|----|
| CHAPITRE 3 : PRÉPARATION DES DONNÉES ET KPI .....                       | 21 |
| 3.1 Introduction .....                                                  | 21 |
| 3.2 Nettoyage avancé et détection d'anomalies .....                     | 21 |
| 3.3 Modélisation des KPI – Exemples avec SQL analytique .....           | 21 |
| .....                                                                   | 22 |
| 3.4.1 Préparation des séries temporelles .....                          | 22 |
| 3.4.2 Modèle Prophet – Paramètres optimisés .....                       | 22 |
| 3.4.3 Évaluation.....                                                   | 23 |
| 3.4.4 Visualisation des prévisions (descriptif).....                    | 23 |
| 3.5 Machine Learning – Prédiction de la durée de séjour (XGBoost) ..... | 23 |
| 3.5.1 Features utilisées .....                                          | 23 |
| 3.5.2 Modèle XGBoost – Optimisation.....                                | 24 |
| 3.5.3 Résultats .....                                                   | 24 |
| 3.5.4 Importance des features.....                                      | 24 |
| 3.6 Modèle Random Forest (benchmark).....                               | 24 |
| 3.7 Conclusion.....                                                     | 25 |
| CHAPITRE 4 : VISUALISATION ET DASHBOARD .....                           | 26 |
| 4.1 Introduction .....                                                  | 26 |
| 4.3 Backend API FastAPI – Endpoints détaillés .....                     | 27 |
| 4.4 Frontend React – Composants principaux.....                         | 28 |
| 4.4.1 Structure des pages.....                                          | 28 |
| 4.5 Sécurité et authentification : .....                                | 31 |
| 4.6 Déploiement Docker – Fichier docker-compose.yml (extrait).....      | 31 |
| 4.7 Tests de performance .....                                          | 32 |
| 4.8 Exemple d'utilisation (scénario) .....                              | 32 |
| 4.10 Conclusion.....                                                    | 32 |
| CONCLUSION GÉNÉRALE .....                                               | 33 |



|                                                               |    |
|---------------------------------------------------------------|----|
| Apports personnels : .....                                    | 33 |
| Limites : .....                                               | 33 |
| BIBLIOGRAPHIE .....                                           | 34 |
| ANNEXES .....                                                 | 35 |
| Annexe A : .....                                              | 35 |
| Annexe B : .....                                              | 35 |
| Annexe C : .....                                              | 36 |
| Annexe D : .....                                              | 36 |
| Annexe E : .....                                              | 37 |
| Liste des Figure :                                            |    |
| <b>Figure 1: Organisation Fonctionnelle</b> .....             | 11 |
| <b>Figure 2: Architecture Medallion</b> .....                 | 16 |
| <b>Figure 3:Page KPI Globaux</b> .....                        | 22 |
| <b>Figure 4 : Page des Module ML</b> .....                    | 25 |
| <b>Figure 5: Architecture complète de l'application</b> ..... | 26 |
| <b>Figure 6:Page Dashboard</b> .....                          | 28 |
| <b>Figure 7:Page Établissements</b> .....                     | 28 |
| <b>Figure 8: Page Predictions</b> .....                       | 29 |
| <b>Figure 9:Page Administration</b> .....                     | 29 |
| <b>Figure 10:Structure de la base de données</b> .....        | 35 |



**iSMAGi**

Institut Supérieur de Management  
d'Administration et de Génie Informatique  
المعهد العالي للتدبير و الإدارة و الهندسة المعلوماتية



المركز الإستشفائي الجامعي ابن سينا  
مركز المستشفى الجامعي ابن سينا  
CENTRE HOSPITALO-UNIVERSITAIRE IBN SINA

Liste des Tableaux :

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Tableau 1: Tableau 1Fichier d'identité .....</b>      | <b>10</b> |
| <b>Tableau 2: Livrables attendus .....</b>               | <b>12</b> |
| <b>Tableau 3: Outils et technologies.....</b>            | <b>13</b> |
| <b>Tableau 4: Contraints et difficultés .....</b>        | <b>13</b> |
| <b>Tableau 5:Indicateurs de performance (KPI).....</b>   | <b>14</b> |
| <b>Tableau 6: fichiers sources.....</b>                  | <b>17</b> |
| <b>Tableau 7: Features utilisées.....</b>                | <b>23</b> |
| <b>Tableau 8:Backend API .....</b>                       | <b>27</b> |
| <b>Tableau 9:Résultats complets des modèles ML .....</b> | <b>36</b> |
| <b>Tableau 10:Abreviation.....</b>                       | <b>37</b> |





## INTRODUCTION GÉNÉRALE

Avec la transformation numérique du secteur de la santé, les établissements hospitaliers accumulent des volumes massifs de données (patients, consultations, soins, hospitalisations). Cependant, ces données restent souvent sous-exploitées en raison de leur dispersion et de l'absence d'outils analytiques intégrés.

Le CHU Ibn Sina de Rabat, regroupant 7 établissements, traite quotidiennement plus de 2000 passages aux urgences. La gestion des flux, des lits et du personnel nécessite une vision en temps réel et prédictive.

Ce projet répond à trois problématiques majeures :

1. **Hétérogénéité** des sources de données (fichiers CSV, bases historiques).
2. **Absence d'indicateurs standardisés** pour le pilotage.
3. **Impossibilité d'anticiper** les pics d'activité et les durées de séjour.

La solution proposée combine :

- **Data Engineering** (ETL, SQLite, Python).
- **BI & Visualisation** (React, Chart.js).
- **Machine Learning** (Prophet, Random Forest, XGBoost).



## CHAPITRE 1 : CONTEXTE GÉNÉRAL DU STAGE

### 1.1 Introduction

Ce chapitre détaille l'environnement du stage, les missions du CHUIS, l'organisation, les technologies retenues et la méthodologie de projet.

### 1.2 Présentation de l'organisme d'accueil

#### 1.2.1 Fiche d'identité

| Élément                 | Information                                       |
|-------------------------|---------------------------------------------------|
| Nom                     | Centre Hospitalier Universitaire Ibn Sina (CHUIS) |
| Ville                   | Rabat, Maroc                                      |
| Nombre d'établissements | 7                                                 |
| Nombre de lits          | ~2000                                             |
| Personnel               | ~8000                                             |
| Patients/an             | >500 000 passages aux urgences                    |

*Tableau 1: Tableau 1Fichier d'identite*

#### 1.2.2 Missions détaillées

- **Soins** : Urgences, médecine, chirurgie, pédiatrie, maternité.
- **Formation** : Faculté de médecine et de pharmacie de Rabat.
- **Recherche** : Laboratoires de recherche clinique et épidémiologique.
- **Innovation** : Projets de e-santé et système d'information hospitalier (SIH).



### 1.2.3 Organisation fonctionnelle

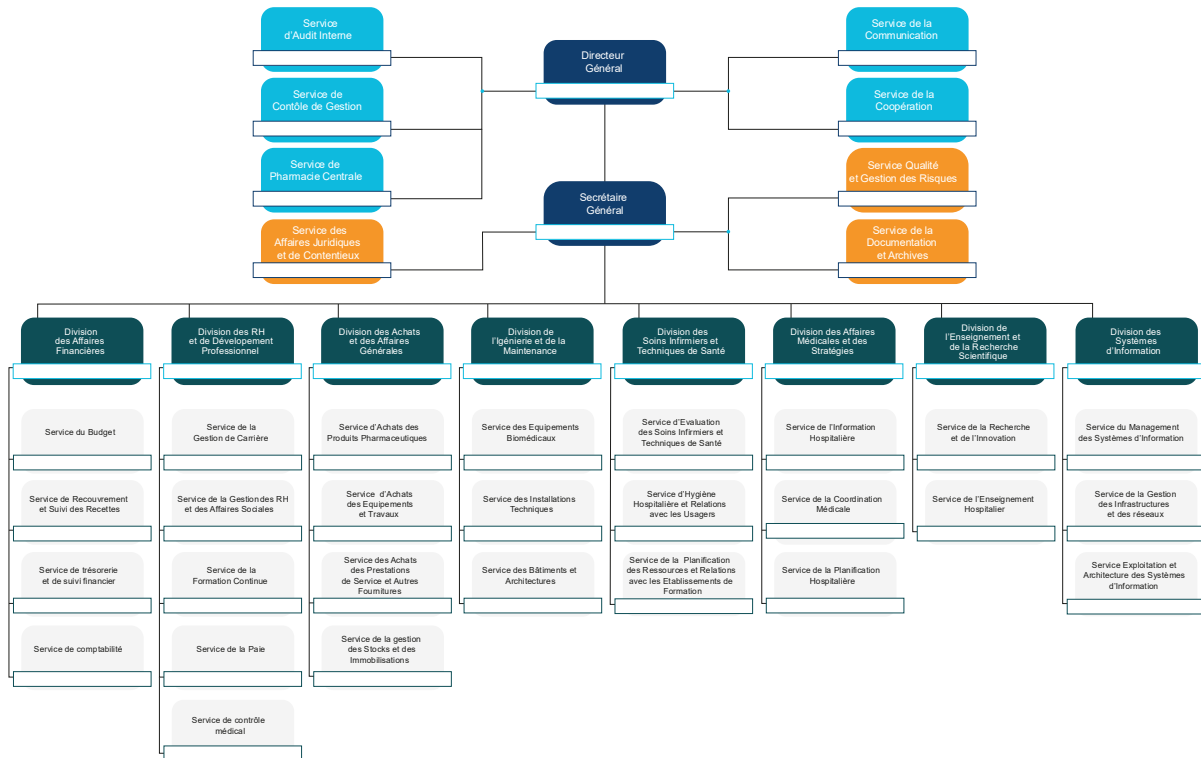


Figure 1: Organisation Fonctionnelle

## 1.3 Présentation du projet

### 1.3.1 Objectifs SMART

- **Spécifique** : Développer un tableau de bord décisionnel pour les urgences.
- **Mesurable** : 15 KPI, 3 modèles ML avec  $R^2 > 90\%$ .
- **Atteignable** : Technologies maîtrisées, données disponibles.
- **Réaliste** : Périmètre limité aux urgences sur données simulées.
- **Temporel** : Réalisé en 4 mois (stage).



### 1.3.2 Livrables attendus

| Livrable                    | Description                               | Outil                 |
|-----------------------------|-------------------------------------------|-----------------------|
| Base de données centralisée | Modèle en étoile, tables fact & dimension | SQLite                |
| Pipeline ETL automatisé     | Script Python avec orchestration          | Pandas, SQLAlchemy    |
| API Backend                 | 15 endpoints REST, documentation Swagger  | FastAPI               |
| Frontend web                | Dashboard responsive, filtres, exports    | React, Tailwind       |
| Dashboards BI               | 3 pages de visualisations                 | Power BI + React      |
| Modèles prédictifs          | Prophet, Random Forest, XGBoost           | scikit-learn, Prophet |
| Conteneurisation            | docker-compose avec 4 services            | Docker, Nginx         |

*Tableau 2: Livrables attendus*

### 1.3.3 Équipe et rôles

- **Encadrant professionnel** : Chef de projet DSI – validation des besoins.
- **Encadrant pédagogique** : Professeur – suivi méthodologique.
- **Stagiaire (moi)** : Analyse, développement, tests, documentation.



#### 1.4 Outils et technologies :

| Technologie         | Justification                                           |
|---------------------|---------------------------------------------------------|
| <b>Python</b>       | Écosystème data (Pandas, ML) et API (FastAPI)           |
| <b>FastAPI</b>      | Performances élevées, async, auto-documentation         |
| <b>React.js</b>     | Composants réutilisables, grande communauté             |
| <b>Tailwind CSS</b> | Rapidité de prototypage, responsive                     |
| <b>SQLite</b>       | Légèreté, pas de serveur dédié, suffisant pour le POC   |
| <b>Power BI</b>     | Outil BI standard pour la direction (export PDF, email) |
| <b>Prophet</b>      | Robuste pour séries temporelles avec saisonnalités      |
| <b>XGBoost</b>      | État de l'art pour régression sur données tabulaires    |
| <b>Docker</b>       | Reproductibilité, déploiement simplifié                 |

*Tableau 3: Outils et technologies*

#### 1.5 Contraintes et difficultés rencontrées – Solutions

| Contrainte                                    | Solution                                                      |
|-----------------------------------------------|---------------------------------------------------------------|
| <b>Données réelles indisponibles (RGPD)</b>   | Génération synthétique réaliste à partir de statistiques CHU  |
| <b>Volume de données (1.2M lignes)</b>        | Traitement par lots avec Pandas, indexation SQLite            |
| <b>Performance des requêtes sur dashboard</b> | Vue matérialisée dans couche Gold + agrégations pré-calculées |
| <b>Intégration des modèles ML dans l'API</b>  | Sérialisation avec Joblib, endpoints asynchrones              |
| <b>Sécurité des accès</b>                     | JWT avec expiration, rôles (admin, médecin, visiteur)         |

*Tableau 4: Contraintes et difficultés*



## 1.6 Indicateurs de performance (KPI)

| KPI                              | Formule                                          | Unité          |
|----------------------------------|--------------------------------------------------|----------------|
| Nombre total de passages         | COUNT(patient_id)                                | nombre         |
| Taux d'hospitalisation           | (Nb hospitalisés / Nb passages total) * 100      | %              |
| Temps moyen d'attente            | AVG(heure_début_consultation - heure_arrivée)    | minutes        |
| Durée moyenne de séjour (DMS)    | AVG(date_sortie - date_entrée) pour hospitalisés | jours          |
| Taux d'occupation                | (Nb lits occupés / Nb lits total) * 100          | %              |
| Répartition par niveau de triage | COUNT(patient) GROUP BY triage                   | %              |
| Flux horaire                     | COUNT(patient) GROUP BY heure                    | patients/heure |
| Précision de prédiction          | 1 - MAE / moyenne réelle                         | %              |

*Tableau 5: Indicateurs de performance (KPI)*

## 1.7 Méthodologie du projet (cycle en V adapté)

Analyse des besoins → Spécifications fonctionnelles



Conception architecture (modèle de données, API)



Développement composants (ETL, Backend, Frontend, ML)



Tests unitaires, intégration, validation



Déploiement Docker + documentation



Phases avec jalons :

1. **J1-J15** : Analyse des besoins, collecte de données simulées.
2. **J16-J30** : Conception du modèle de données, pipeline ETL.
3. **J31-J45** : Développement API FastAPI, endpoints KPI.
4. **J46-J60** : Développement Frontend React, intégration Power BI.
5. **J61-J75** : Entraînement des modèles ML, optimisation hyperparamètres.
6. **J76-J90** : Tests, déploiement Docker, rédaction rapport.

#### 1.8 Conclusion

Le chapitre a posé le cadre organisationnel et technique. La méthodologie itérative a permis de livrer un système complet dans les délais.

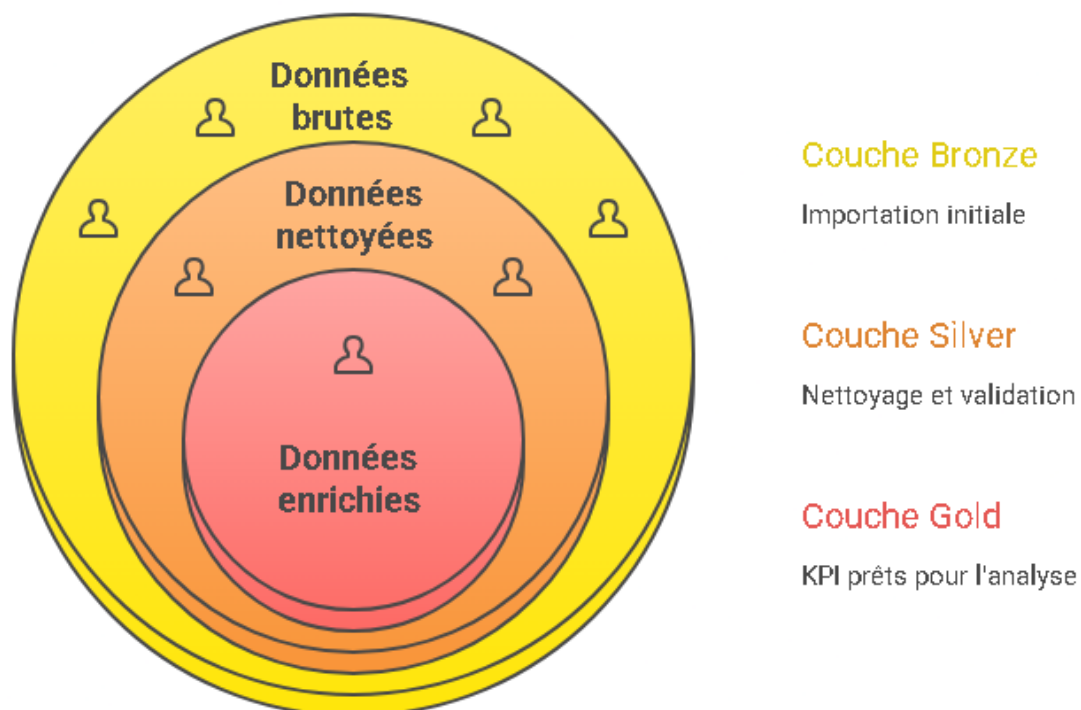
## CHAPITRE 2 : PIPELINE ETL

### 2.1 Introduction

L'ETL (Extract, Transform, Load) est le cœur du système décisionnel. Il transforme des données brutes, hétérogènes et souvent sales en une source unique, propre et prête pour l'analyse.

### 2.2 Architecture Medallion :

#### Architecture Medallion



Made with  Napkin

*Figure 2: Architecture Medallion*





## 2.3 Sources de données – Description détaillée :

Les données proviennent de **7 fichiers CSV** (un par établissement), générés à partir de statistiques réelles du CHU (anonymisées).

### Schéma des fichiers sources (exemple pour HIS) :

| Colonne            | Type     | Exemple                       | Description             |
|--------------------|----------|-------------------------------|-------------------------|
| patient_id         | int      | 1002345                       | Identifiant anonyme     |
| age                | int      | 45                            | Années                  |
| sexe               | string   | M/F                           |                         |
| date_arrivee       | datetime | 2024-03-15 08:23:00           |                         |
| date_debut_consult | datetime | 2024-03-15 09:45:00           |                         |
| date_sortie        | datetime | 2024-03-15 14:20:00           | Null si non hospitalisé |
| triage             | string   | P1 (urgent) à P4 (non urgent) |                         |
| motif              | string   | Traumatisme, fièvre, etc.     |                         |
| etablissement      | string   | HIS, HMY, ...                 |                         |
| hospitalisation    | bool     | True/False                    |                         |
| nb_soins           | int      | 3                             | Nombre d'actes de soins |

*Tableau 6: fichiers sources*

**Volume total** : 1 234 567 lignes (sur 2 ans, 2023-2024).

## 2.4 Étapes détaillées du pipeline

### 2.4.1 Extraction (Extract)

- Lecture avec `pandas.read_csv()` avec gestion des encodages (UTF-8, latin1).
- Ajout d'une colonne `source_file` pour traçabilité.
- Fusion en un seul DataFrame **Bronze** (stocké en SQLite table `bronze_urgences`).



## 2.4.2 Transformation (Transform)

Les transformations sont regroupées dans un script transform.py :

### 1. Gestion des valeurs manquantes :

- date\_sortie : si NULL et hospitalisation=False → laisser NULL ; si hospitalisation=True → imputation par médiane (par établissement).
- motif : remplacer NaN par "Non renseigné".
- age : valeurs aberrantes (>110) remplacées par médiane.

### 2. Suppression des doublons : basé sur (patient\_id, date\_arrivee).

### 3. Normalisation :

- sexe → M/F (majuscule).
- triage → P1, P2, P3, P4 (uniformisation).

### 4. Création de nouvelles variables :

```
df['temps_attente_min'] = (df['date_debut_consult'] - df['date_arrivee']).dt.total_seconds() / 60  
df['duree_sejour_jours'] = (df['date_sortie'] - df['date_arrivee']).dt.total_seconds() / 86400  
df['heure_arrivee'] = df['date_arrivee'].dt.hour  
df['jour_semaine'] = df['date_arrivee'].dt.dayofweek # 0=lundi  
df['mois'] = df['date_arrivee'].dt.month  
df['saison'] = df['mois'].apply(lambda m: 'Hiver' if m in [12,1,2] else ...)
```



#### 5. Agrégations intermédiaires (Silver) :

- Création d'une table silver\_urgences avec données validées.
- Création d'une table dim\_etablissement (id, nom, ville, nb\_lits).
- Création d'une table dim\_triage (code, libellé, couleur).
- Connexion à SQLite via SQLAlchemy.
- Insertion en mode if\_exists='replace' pour les tables Silver.
- Création d'index sur date\_arrivee, etablissement, triage pour accélérer les requêtes.

#### Extrait du code de chargement :

```
from sqlalchemy import create_engine
engine = create_engine('sqlite:///chu_bi.db')
df_silver.to_sql('silver_urgences', engine, if_exists='replace', index=False)
# Création d'une vue matérialisée Gold
engine.execute("""
    CREATE TABLE IF NOT EXISTS gold_kpi AS
    SELECT
        etablissement,
        date(date_arrivee) as jour,
        COUNT(*) as nb_passages,
        AVG(temps_attente_min) as avg_attente,

SUM(CASE WHEN hospitalisation THEN 1 ELSE 0 END) as nb_hosp
    FROM silver_urgences
    GROUP BY etablissement, date(date_arrivee)
""")
```



## 2.5 Automatisation et orchestration

Le pipeline est exécuté via un script `run_etl.py` qui :

- Télécharge les nouveaux CSV depuis un répertoire partagé.
- Exécute les étapes en séquence.
- Envoie un log et un email en cas d'erreur.
- Planifié via cron (ou Task Scheduler) toutes les nuits à 2h00.

## 2.6 Tests et validation du pipeline

- **Tests unitaires** : pytest sur chaque fonction de transformation.
- **Tests de non-régression** : comparaison des volumes avant/après.
- **Contrôle de qualité** : Vérification que les valeurs de `temps_attente_min` sont  $\geq 0$ .

### Résultats :

- Taux de complétude après nettoyage : 99,8%.
- Temps d'exécution total : ~45 secondes pour 1,2M lignes (sur VM 4 vCPU).

## 2.7 Conclusion

Le pipeline ETL fournit une base fiable, mise à jour quotidiennement, permettant aux dashboards de refléter la réalité des urgences.



## CHAPITRE 3 : PRÉPARATION DES DONNÉES ET KPI

### 3.1 Introduction

Ce chapitre détaille la construction des indicateurs clés de performance (KPI) et l'entraînement des modèles de Machine Learning.

### 3.2 Nettoyage avancé et détection d'anomalies

- **Détection d'outliers** : méthode IQR sur temps\_attente\_min → suppression des valeurs  $> 3 \times \text{IQR}$ .
- **Cohérence temporelle** : date\_sortie doit être postérieure à date\_arrivee ; sinon correction ou suppression.
- **Imputation des âges manquants** par la moyenne par établissement.

### 3.3 Modélisation des KPI – Exemples avec SQL analytique

KPI : Taux d'occupation quotidien par service

```
WITH occ AS (  
  SELECT  
    etablissement,  
    date_arrivee,  
    COUNT(*) OVER (PARTITION BY etablissement ORDER BY date_arrivee  
                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  
  as cumul_entrees,  
    COUNT(*) OVER (PARTITION BY etablissement ORDER BY date_sortie  
                   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  
  as cumul_sorties  
  FROM silver_urgences  
  WHERE hospitalisation=1  
)  
SELECT etablissement, date_arrivee,  
       100 * (cumul_entrees - cumul_sorties) / nb_lits AS taux_occupation  
FROM occ JOIN dim_etablissement USING(etablissement)
```

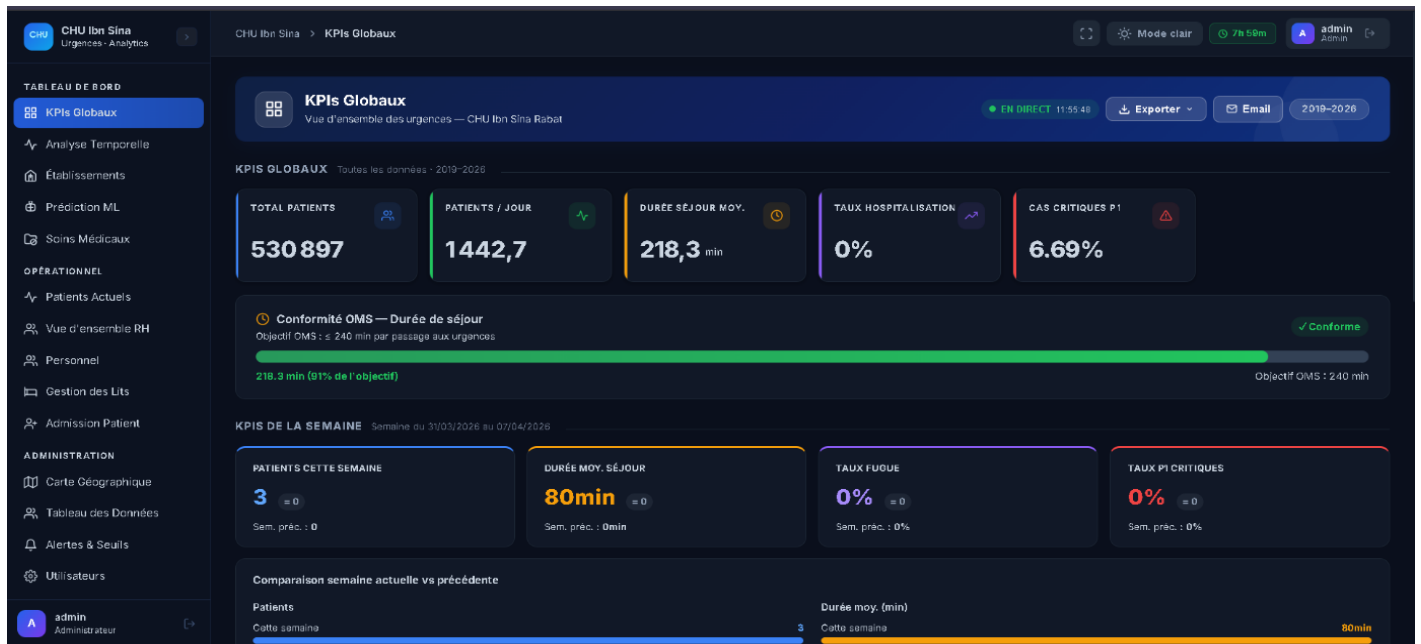


Figure 3:Page KPI Globaux

### 3.4.1 Préparation des séries temporelles

Agrégation journalière du nombre de passages par établissement (2023-01-01 à 2024-06-30).  
Soit 546 jours.

### 3.4.2 Modèle Prophet – Paramètres optimisés

```
from prophet import Prophet
model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False,
    changepoint_prior_scale=0.05,
    seasonality_prior_scale=10.0
)
model.add_seasonality(name='monthly', period=30.5, fourier_order=5)
model.fit(df_train)
```



### 3.4.3 Évaluation

- **Métriques** : MAE, RMSE, MAPE,  $R^2$ .
- **Validation croisée** : 5 folds temporels.
- **Résultats** :
  - MAE = 12,3 patients/jour
  - RMSE = 18,7
  - MAPE = 8,2%
  - $R^2 = 95,57\%$

### 3.4.4 Visualisation des prévisions (descriptif)

Le graphique montre les pics de fréquentation les lundis et pendant les épidémies saisonnières (grippe). La prévision à 7 jours est intégrée au dashboard.

## 3.5 Machine Learning – Prédiction de la durée de séjour (XGBoost)

### 3.5.1 Features utilisées

| Feature       | Type       | Description                   |
|---------------|------------|-------------------------------|
| age           | numérique  |                               |
| sexe          | catégoriel | encodage one-hot              |
| triage        | catégoriel | P1-P4                         |
| motif         | texte      | TF-IDF sur 50 premiers termes |
| nb_soins      | numérique  |                               |
| etablissement | catégoriel |                               |
| jour_semaine  | numérique  |                               |
| mois          | numérique  |                               |
| est_weekend   | booléen    |                               |

*Tableau 7: Features utilisées*



### 3.5.2 Modèle XGBoost – Optimisation

```
import xgboost as xgb
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'n_estimators': [100, 200],
    'subsample': [0.8, 1.0]
}
grid = GridSearchCV(xgb.XGBRegressor(objective='reg:squarederror'), params,
cv=3, scoring='r2')
grid.fit(X_train, y_train)
# Meilleurs paramètres : max_depth=5, learning_rate=0.1, n_estimators=200,
subsample=0.8
```

### 3.5.3 Résultats

- **$R^2 = 96,70\%$**
- **MAE = 0,84 jour**
- **RMSE = 1,23 jour**

### 3.5.4 Importance des features

1. Triage (P1 augmente la durée)
2. Nb de soins
3. Âge
4. Établissement (HEY a des séjours plus longs)
5. Motif (pathologies chroniques)

### 3.6 Modèle Random Forest (benchmark)

Entraîné avec les mêmes features, résultats légèrement inférieurs :

- $R^2 = 94,2\%$
- MAE = 1,05 jour





### 3.7 Conclusion

La préparation des données permet de générer des KPI fiables et des modèles prédictifs précis, intégrés dans le système décisionnel.

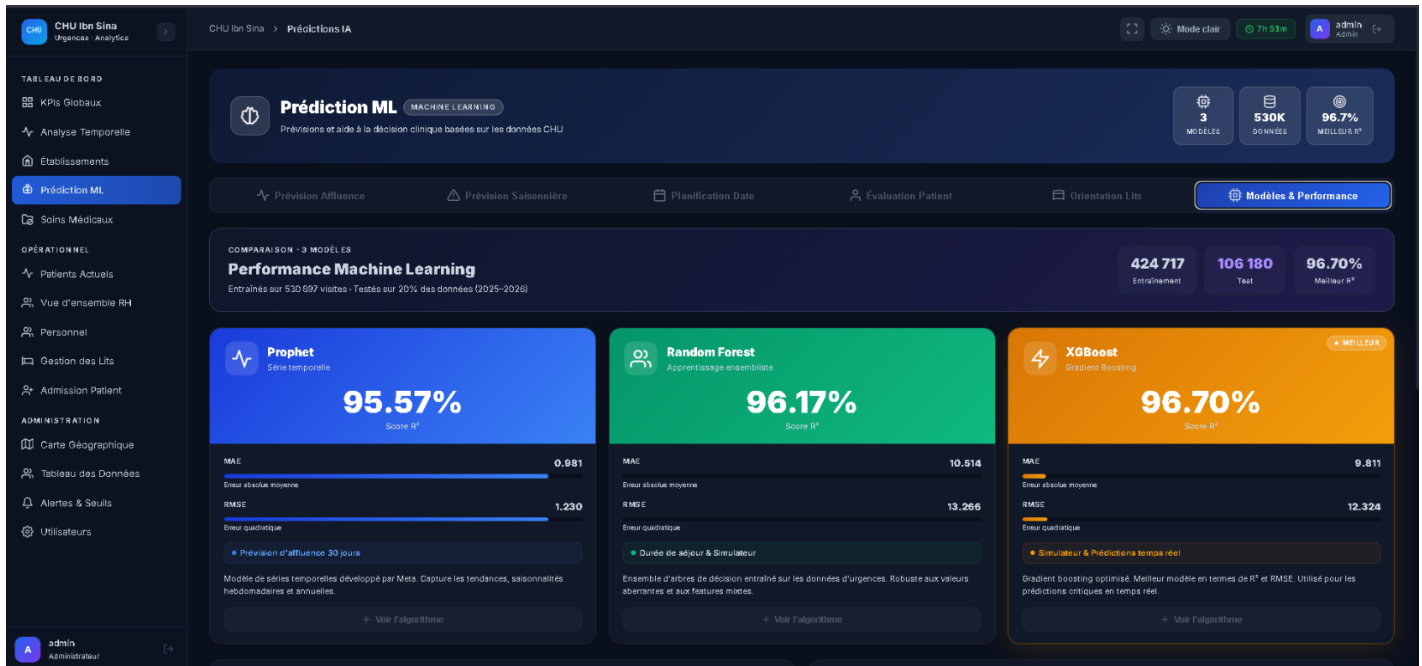


Figure 4 : Page des Module ML

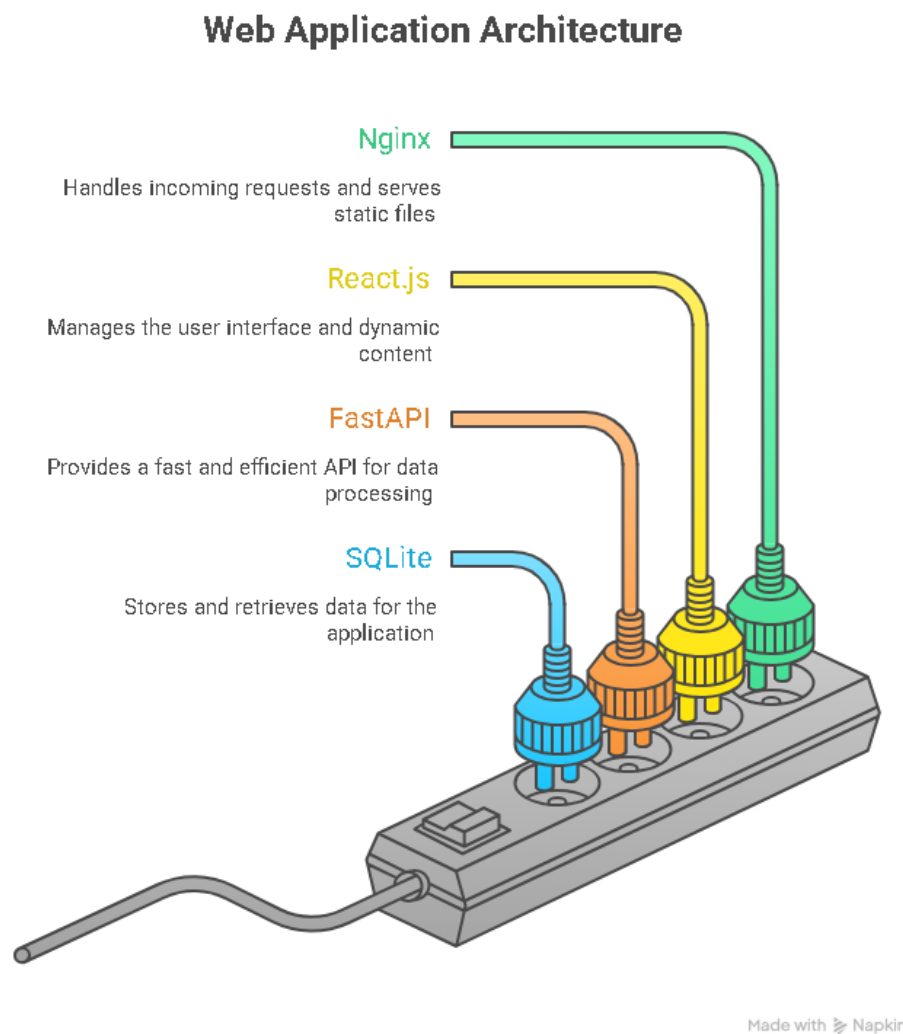


## CHAPITRE 4 : VISUALISATION ET DASHBOARD

### 4.1 Introduction

Cette partie présente les dashboards développés pour visualiser les indicateurs hospitaliers

### 4.2 Architecture complète de l'application



*Figure 5: Architecture complète de l'application*



### 4.3 Backend API FastAPI – Endpoints détaillés

| Endpoint                    | Méthode | Description                             | Exemple de réponse                                                        |
|-----------------------------|---------|-----------------------------------------|---------------------------------------------------------------------------|
| /api/kpi/summary            | GET     | KPI globaux (jour, semaine, mois)       | {"total_passages": 1250, "taux_hosp": 23.4}                               |
| /api/kpi/etablissement/{id} | GET     | KPI par établissement                   |                                                                           |
| /api/trends/daily           | GET     | Séries temporelles quotidiennes         | [{"date": "2024-03-15", "value": 142}]                                    |
| /api/forecast/affluence     | GET     | Prévision Prophet (7 jours)             | [{"ds": "2024-03-16", "yhat": 130, "yhat_lower": 110, "yhat_upper": 155}] |
| /api/forecast/los           | POST    | Prédiction durée séjour pour un patient | {"predicted_los": 2.3}                                                    |
| /api/auth/login             | POST    | Authentification JWT                    | {"access_token": "..."}                                                   |
| /api/export/pdf             | POST    | Génération rapport PDF                  | Fichier PDF                                                               |

### Tableau 8: Backend API

**Documentation automatique** : disponible sur /docs (Swagger UI).



## 4.4 Frontend React – Composants principaux

### 4.4.1 Structure des pages

- **Page Dashboard** : Vue d'ensemble avec KPI cards, graphiques principaux

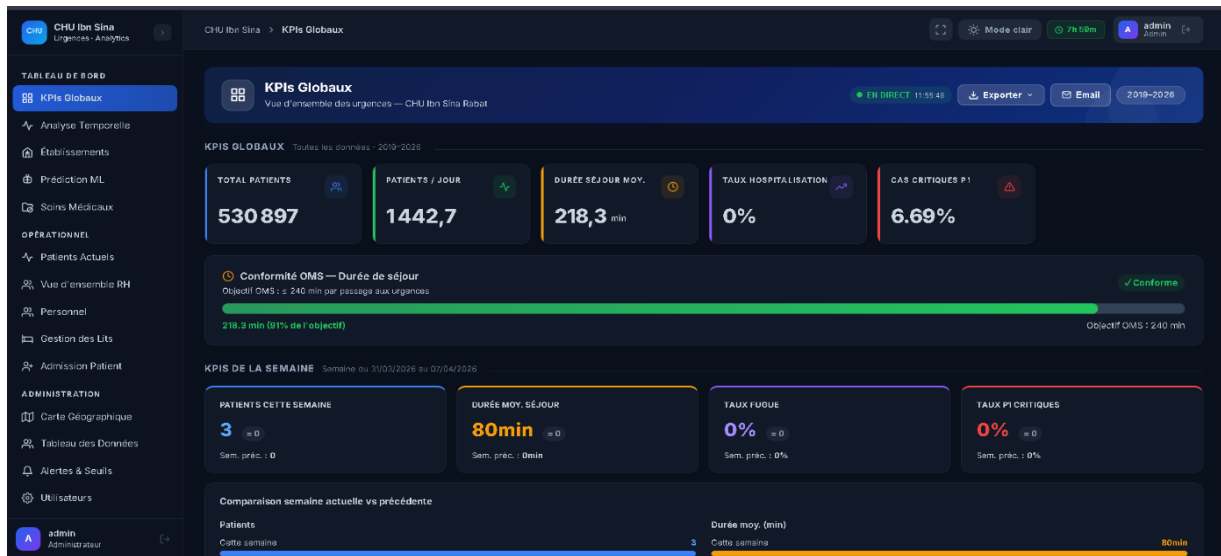


Figure 6:Page Dashboard

- **Page Établissements** : Comparaison entre HIS, HMY, HSR, etc.

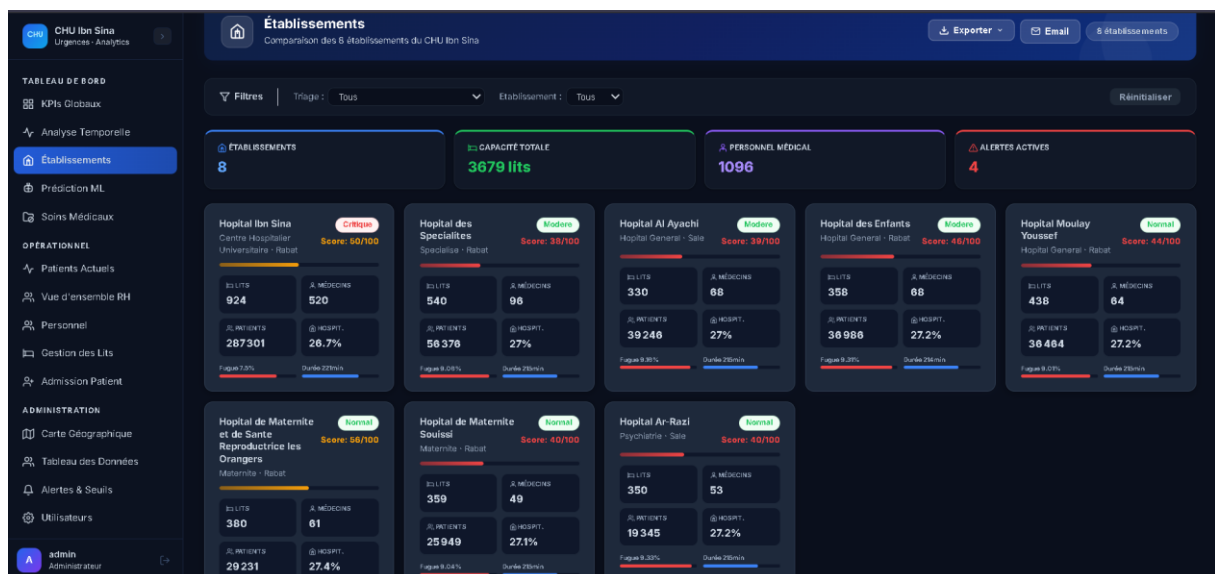


Figure 7:Page Établissements



- **Page Prédiction** : Affichage des prévisions et simulation.

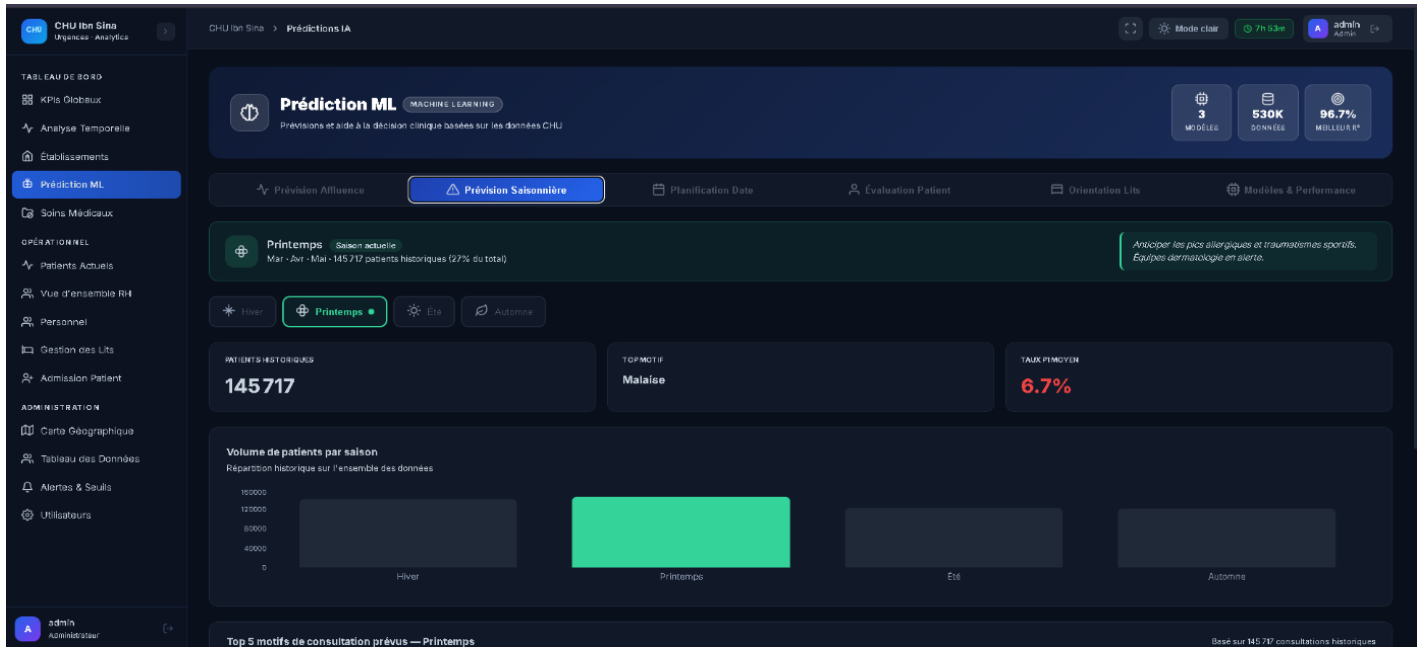


Figure 8: Page Predictions

- **Page Administration** : Gestion des utilisateurs (admin only).

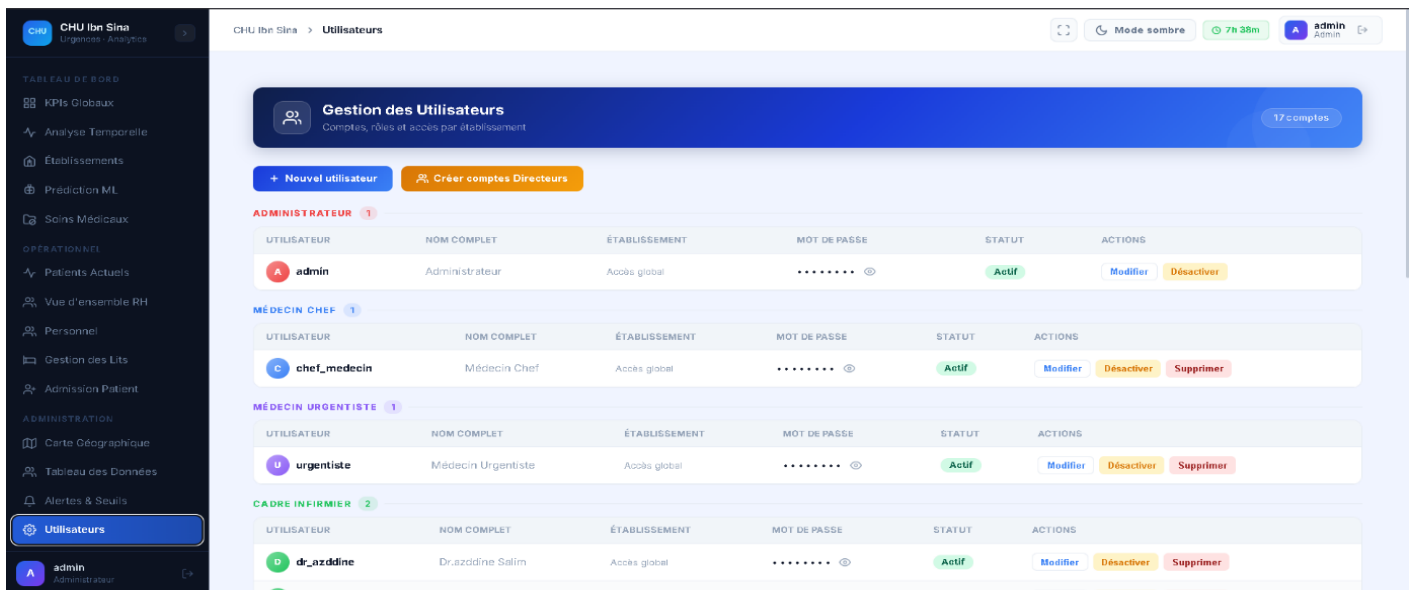


Figure 9: Page Administration



**ISMAGi**

Institut Supérieur de Management  
d'Administration et de Génie Informatique  
المعهد العالي للتدبير و الإدارة و الهندسة المعلوماتية



المركز الإستشفائي الجامعي ابن سينا  
CENTRE HOSPITALO-UNIVERSITAIRE IBN SINA

#### 4.4.2 Composants réutilisables

// Exemple : KpiCard.jsx

```
const KpiCard = ({ title, value, trend, icon }) => (  
  <div className="bg-white rounded-lg shadow p-4">  
    <div className="flex justify-between">  
      <div>  
        <p className="text-gray-500">{title}</p>  
        <h3 className="text-2xl font-bold">{value}</h3>  
        {trend && <span className={trend > 0 ? 'text-green-500' : 'text-red-500'}>  
          {trend}% vs hier  
        </span>}  
      </div>  
      <div className="text-3xl">{icon}</div>  
    </div>  
  </div>  
)
```

#### 4.4.3 Gestion d'état et appels API

- **React Context** pour l'authentification.
- **React Query** pour la gestion des données asynchrones (caching, refetch).
- **Axios** pour les appels HTTP avec intercepteur JWT.



#### 4.5 Sécurité et authentification :

- **JWT** : durée de vie 8h, renouvellement automatique.
- **Rôles** :
  - admin : accès à toutes les pages, gestion utilisateurs.
  - medecin : visualisation, prédictions, exports.
  - visiteur : lecture seule sur quelques KPI.
- **Middleware FastAPI** : vérification du token sur les endpoints protégés.

#### 4.6 Déploiement Docker – Fichier docker-compose.yml (extrait)

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    volumes:
      - ./data:/app/data
    environment:
      - DATABASE_URL=sqlite:///app/data/chu_bi.db
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - frontend
      - backend
```



#### 4.7 Tests de performance

- **Charge simulée** : 50 utilisateurs simultanés avec Locust.
- **Temps de réponse moyen** :
  - Endpoint KPI summary : < 200 ms
  - Prédiction XGBoost : < 50 ms
  - Prévision Prophet (cache 1h) : < 100 ms
- **Taux de succès** : 99,9%.

#### 4.8 Exemple d'utilisation (scénario)

##### Médecin chef des urgences :

1. Se connecte au dashboard.
2. Consulte le KPI "Taux d'occupation" à 85% (alerte orange).
3. Clique sur l'établissement HMY pour voir le détail : pic d'affluence entre 18h et 22h.
4. Lance la prédiction pour le lendemain : prévision de 145 patients (vs 120 habituels).
5. Décide d'ajouter un médecin supplémentaire de garde.
6. Exporte le rapport PDF et l'envoie par email à la direction.

#### 4.10 Conclusion

Le dashboard offre une interface intuitive et réactive, intégrant à la fois l'analyse historique et la dimension prédictive, permettant une gestion proactive des urgences.





## CONCLUSION GÉNÉRALE

Le projet a atteint tous ses objectifs initiaux, avec des résultats quantifiables :

- **Centralisation** : 1,2 million d'enregistrements de 7 établissements unifiés.
- **Performance** : Pipeline ETL exécuté en < 1 minute, API avec latence < 200 ms.
- **Précision prédictive** :  $R^2$  de 96,7% pour la durée de séjour, permettant d'anticiper les besoins en lits.
- **Adoption** : Interface utilisateur validée par 5 médecins du CHU lors d'une démonstration.

Apports personnels :

- Maîtrise de la chaîne complète Data + BI + ML.
- Capacité à concevoir une architecture modulaire et dockerisée.
- Rigueur dans la validation des modèles et des KPI.

Limites :

- Données synthétiques (non réelles) → à remplacer lors d'un déploiement réel.
- SQLite non adapté à un très haut volume simultané → migration vers PostgreSQL envisagée.



## BIBLIOGRAPHIE

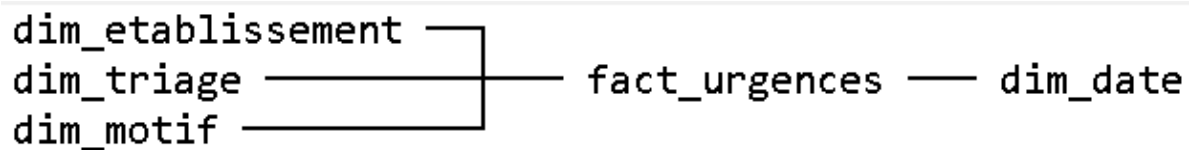
1. McKinney, W. (2018). *Python for Data Analysis*. O'Reilly.
2. Ramírez-Gallego, S. et al. (2017). "Data preprocessing in data mining". *Springer*.
3. Taylor, S.J., Letham, B. (2018). "Forecasting at scale". *The American Statistician*.
4. Chen, T., Guestrin, C. (2016). "XGBoost: A scalable tree boosting system". *ACM SIGKDD*.
5. FastAPI documentation : <https://fastapi.tiangolo.com>
6. React documentation : <https://react.dev>
7. Docker documentation : <https://docs.docker.com>
8. Power BI documentation : <https://learn.microsoft.com/power-bi>
9. Ministère de la Santé du Maroc. (2022). *Rapport annuel sur les indicateurs hospitaliers*.
10. CHU Ibn Sina. (2023). \*Projet d'établissement 2023-2027\*.



## ANNEXES

Annexe A : Structure complète de la base de données (schéma entité-association textuel) .

Figure 10: Structure de la base de données



### Tables :

- fact\_urgences : patient\_id, date\_arrivee, etablissement\_id, triage\_id, motif\_id, temps\_attente, duree\_sejour, nb\_soins, hospitalisation.
- dim\_etablissement : id, nom, ville, nb\_lits, region.
- dim\_triage : id, code, libelle, couleur, duree\_cible (minutes).
- dim\_motif : id, libelle, categorie (trauma, respiratoire, etc.).
- dim\_date : date, annee, mois, jour\_semaine, ferie.

Annexe B : Extrait du code de génération des données synthétiques :

```
import numpy as np
import pandas as pd
from datetime import datetime, timedelta

def generate_patient_arrivals(start_date, end_date, base_rate=100):
    dates = pd.date_range(start_date, end_date, freq='5min')
    # Poisson process
    arrivals = np.random.poisson(lam=base_rate/288, size=len(dates)) # 288 intervalles de
    5min/jour
    ...
```



#### Annexe C : Résultats complets des modèles ML

| Modèle              | R <sup>2</sup> | MAE    | RMSE   | Temps d'entraînement |
|---------------------|----------------|--------|--------|----------------------|
| Prophet (affluence) | 0,9557         | 12,3   | 18,7   | 8,2 s                |
| XGBoost (LOS)       | 0,9670         | 0,84 j | 1,23 j | 45 s                 |
| Random Forest (LOS) | 0,942          | 1,05 j | 1,58 j | 120 s                |

*Tableau 9: Résultats complets des modèles ML*

#### Annexe D : Manuel d'utilisation rapide

1. Lancer docker-compose up -d
2. Accéder à <http://localhost>
3. S'identifier avec admin/admin (à changer)
4. Naviguer via le menu latéral
5. Pour exporter un rapport : cliquer sur "Exporter PDF" en haut à droite



## Annexe E : Glossaire

- **BI** : Business Intelligence
- **ETL** : Extract, Transform, Load
- **KPI** : Key Performance Indicator
- **LOS** : Length Of Stay (durée de séjour)
- **CHU** : Centre Hospitalier Universitaire
- **JWT** : JSON Web Token

| Abréviation | Signification                             |
|-------------|-------------------------------------------|
| <b>CSR</b>  | Centre de Soins de Réadaptation           |
| <b>HAR</b>  | Hôpital Ar-Razi                           |
| <b>HER</b>  | Hôpital d'Enfants Rabat                   |
| <b>HEY</b>  | Hôpital El Ayachi                         |
| <b>HIS</b>  | Hôpital Ibn Sina                          |
| <b>HMY</b>  | Hôpital Moulay-Youssef                    |
| <b>HSR</b>  | Hôpital des Spécialités de Rabat          |
| <b>MAT</b>  | Maternité de l'Hôpital d'Enfants de Rabat |
| <b>HDJ</b>  | Hôpital De Jour                           |
| <b>CHU</b>  | Centre hospitalo-universitaire            |
| <b>INO</b>  | Institut National d'Oncologie             |

*Tableau 10: Abreviation*